

Prometheus - Alertmanager - Slack Entegrasyonu

Mimari, alarm kataloğu, güvenli webhook yönetimi, doğrulama, günlük işletim ve production'a geçiş rehberi

DURUM	AKTİF VE DOĞRULANMIŞ
BİLDİRİM	SLACK / #alerts
SÜRÜM	Alertmanager v0.32.1
TARİH	4 Ağustos 2026

İçindekiler

Doküman bilgileri	4
1. Amaç ve kapsam	4
2. Mimari	5
Bileşen sorumlulukları	5
3. Dosya yapısı	5
4. Çalışma zamanı topolojisi	6
5. Alarm yaşam döngüsü	6
6. Prometheus yapılandırması	7
Global ayarlar	7
Scrape hedefleri	7
Rule ve Alertmanager bağlantısı	7
7. Alertmanager routing yapısı	7
Ortak grupta	7
Severity routing	8
Inhibition kuralları	8
8. Alarm kataloğu	8
Uygulama alarmları	8
Monitoring alarmları	9
9. Slack entegrasyonu ve secret yönetimi	9
Webhook oluşturma	9
Secret dosyasını hazırlama	9
10. Kurulum ve başlatma	10
Ön koşullar	10
Servisleri başlatma	10
Çalışma durumunu kontrol etme	10
11. Doğrulama ve test	10
Compose şeması	10
Prometheus config ve rule kontrolü	10
Prometheus rule unit testleri	10
Alertmanager config ve template kontrolü	10

Uçtan uca Slack testi	10
12. Günlük işletim	11
UI adresleri	11
Log inceleme	11
Prometheus hedeflerini kontrol etme	11
Aktif alarmları listeleme	11
Planlı bakım ve silence	11
13. Değişiklik yönetimi	11
Alarm kuralı değiştirirken	11
Alertmanager route veya template değiştirirken	12
14. Sorun giderme	12
Önerilen teşhis sırası	12
15. Güvenlik notları	12
16. Kalıcılık ve veri davranışı	13
17. Mevcut sınırlamalar	13
18. Production'a geçiş kontrol listesi	13
19. Bildirimleri geçici durdurma ve geri dönüş	14
20. Kabul testi sonuçları	14
21. Resmî referanslar	14

Doküman bilgileri

Alan	Değer
Durum	Aktif ve doğrulanmış
Çalışma modeli	Docker Compose
Bildirim kanalı	Finovation Slack workspace / #alerts
Alertmanager sürümü	v0.32.1
Ortam etiketi	local
Son doğrulama	4 Ağustos 2026

Bu doküman Finovation projesindeki Prometheus, Alertmanager ve Slack alarm entegrasyonunun mimarisini, yapılandırmasını, alarm kataloğunu, işletim prosedürlerini ve production'a geçiş gereksinimlerini açıklar.

1. Amaç ve kapsam

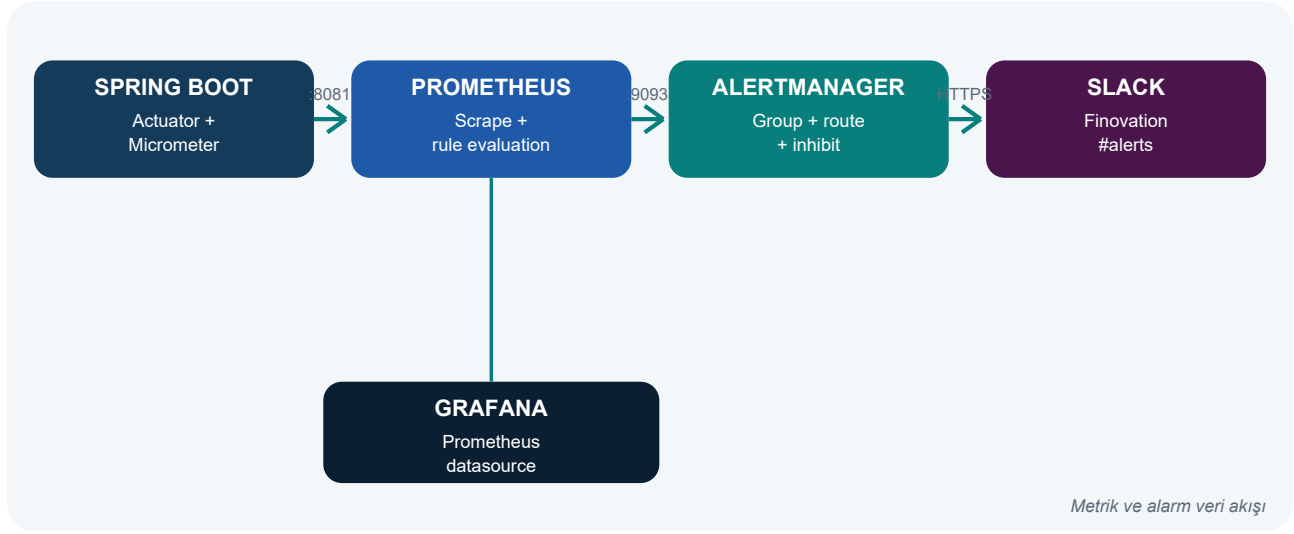
Entegrasyonun amacı, backend ve monitoring bileşenlerinde oluşan operasyonel problemleri Prometheus kurallarıyla tespit etmek, Alertmanager ile gruplayıp önceliklendirmek ve ilgili bildirimleri Slack [#alerts](#) kanalına göndermektir.

Mevcut kapsam şunları içerir:

- Backend erişilebilirliği
- HTTP 5xx hata oranı
- HikariCP bağlantı havuzu doygunluğu
- JVM heap ve process CPU kullanımı
- Prometheus ve Alertmanager self-monitoring alarmları
- [critical](#) ve [warning](#) seviyelerinin Slack'e yönlendirilmesi
- Alarm gruplama, tekrar aralığı, resolved bildirimi ve inhibition
- Config doğrulama ve temel Prometheus rule unit testleri

Mevcut kapsam frontend, Redis ve SQL Server için bağımsız exporter/probe veya uygulamaya özgü auth/mail metriklerini henüz içermez. Bu başlıklar production iyileştirmeleri bölümünde ele alınmıştır.

2. Mimari



Bileşen sorumlulukları

Bileşen	Sorumluluk
Spring Boot / Micrometer	JVM, HTTP ve HikariCP metriklerini üretir.
Prometheus	Metrikleri 15 saniyede bir toplar ve alarm kurallarını 15 saniyede bir değerlendirir.
Alertmanager	Alarmları gruplayıp tekilleştirir, inhibition uygular ve Slack'e gönderir.
Slack	Operasyon ekibinin #alerts kanalında bildirim almasını sağlar.
Grafana	Prometheus metriklerini dashboard üzerinden görselleştirir; alarm kurallarının kaynağı değildir.

3. Dosya yapısı

```

monitoring/
├── prometheus/
│   ├── prometheus.yml
│   ├── rules/
│   │   ├── application.rules.yml
│   │   └── monitoring.rules.yml
│   └── tests/
│       └── application.rules.test.yml
├── alertmanager/
│   ├── alertmanager.yml
│   ├── README.md
│   ├── templates/
│   │   └── slack.tpl
│   └── secrets/
│       ├── .gitkeep
│       ├── slack_webhook_url.example
│       └── slack_webhook_url # Git tarafından ignore edilir

```

Dosya	Açıklama
docker-compose.yml	Prometheus, Alertmanager, Grafana servisleri, volume ve secret bağlantıları
monitoring/prometheus/prometheus.yml	Scrape hedefleri, rule dosyaları ve Alertmanager hedefi
monitoring/prometheus/rules/application.rules.yml	Backend/JVM/HikariCP alarmları

Dosya	Açıklama
<code>monitoring/prometheus/rules/monitoring.rules.yml</code>	Prometheus ve Alertmanager self-monitoring alarmları
<code>monitoring/prometheus/tests/application.rules.test.yml</code>	Kritik Prometheus rule davranış testleri
<code>monitoring/alertmanager/alertmanager.yml</code>	Routing, receiver, Slack ve inhibition ayarları
<code>monitoring/alertmanager/templates/slack.tpl</code>	Slack mesaj başlığı ve gövde şablonu
<code>monitoring/alertmanager/secrets/slack_webhook_url</code>	Gerçek Slack Incoming Webhook URL'si; repoya eklenmez

4. Çalışma zamanı topolojisi

Servis	Container	Ağ adresi	Host erişimi	Kalıcı veri
Backend	<code>finovation-backend</code>	<code>backend:8080</code> , <code>management</code> <code>backend:8081</code>	<code>localhost:8080</code>	Uygulama dışı
Prometheus	<code>finovation-prometheus</code>	<code>prometheus:9090</code>	Host'a publish edilmez	<code>finovation_prometheus_data</code>
Alertmanager	<code>finovation-alertmanager</code>	<code>alertmanager:9093</code>	Yalnızca <code>127.0.0.1:9093</code>	<code>finovation_alertmanager_data</code>
Grafana	<code>finovation-grafana</code>	<code>grafana:3000</code>	<code>localhost:3000</code>	<code>finovation_grafana_data</code>

Alertmanager UI'nin `127.0.0.1` ile sınırlandırılması, silence ve alarm yönetim arayüzünün yerel makine dışından doğrudan erişilmesini engeller.

5. Alarm yaşam döngüsü

- Prometheus ilgili PromQL ifadesini her 15 saniyede bir değerlendirir.
- İfade doğru olduğunda alarm `pending` durumuna geçer.
- Koşul `for` süresi boyunca devam ederse alarm `firing` olur.
- Prometheus alarmı Alertmanager'a gönderir.
- Alertmanager alarmı etiketlerine göre gruplar ve route seçer.
- `critical` veya `warning` alarm Slack `#alerts` kanalına gönderilir.
- Koşul düzeldiğinde `keep_firing_for` süresi varsa bu süre tamamlanır.
- Alarm resolved olur ve `send_resolved: true` nedeniyle Slack'e iyileşme

bildirimi gönderilir.

`for` kısa süreli sıçramaların bildirim üretmesini, `keep_firing_for` ise alarmin hızlı biçimde firing/resolved arasında gidip gelmesini azaltır.

6. Prometheus yapılandırması

Global ayarlar

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    environment: "local"
```

- **scrape_interval**: Metrik toplama sıklığıdır.
- **evaluation_interval**: Alarm kurallarının değerlendirilme sıklığıdır.
- **environment**: Alertmanager routing ve gruplamada kullanılan ortam etiketidir.

Production veya staging dağıtımından önce **environment** değeri ilgili ortamla değiştirilmelidir.

Scrape hedefleri

Job	Hedef	Ek etiketler
finovation-backend	backend:8081/actuator/prometheus	service=finovation-backend, component=api
prometheus	prometheus:9090/metrics	service=finovation-monitoring, component=prometheus
alertmanager	alertmanager:9093/metrics	service=finovation-monitoring, component=alertmanager

Rule ve Alertmanager bağlantısı

Prometheus `/etc/prometheus/rules/*.rules.yml` altındaki dosyaları yükler ve firing alarmları **alertmanager:9093** hedefine gönderir. Rule dosyaları Docker Compose ile read-only olarak container'a bağlanır.

7. Alertmanager routing yapısı

Ortak gruplama

```
group_by: [environment, service, component, alertname]
group_wait: 30s
group_interval: 5m
repeat_interval: 4h
```

- Aynı ortam, servis, component ve alarm adına sahip alarmlar tek Slack

bildirimi altında gruplanır.

- **instance** grouping'e dahil edilmez; çoklu instance probleminde bildirim

fırtınası oluşması önlenir.

- İlk warning bildirimi en fazla 30 saniye gruplanmayı bekler.

Severity routing

Severity	Receiver	İlk bekleme	Tekrar aralığı	Kullanım amacı
critical	slack-alerts	10 saniye	1 saat	Kullanıcı akışını veya alarm hattını kesen kritik sorunlar
warning	slack-alerts	30 saniye	4 saat	Kapasite, performans veya kalıcı bozulma sinyalleri
Diğer / info	null	-	-	Dış bildirim üretmeyen kayıtlar

Inhibition kuralları

İki inhibition davranışı tanımlıdır:

- Aynı `environment`, `service`, `component` ve `alertname` için critical alarm

firing durumundayken warning kopyası bildirilmez.

- `BackendDown` firing durumundayken aynı ortam ve servise ait warning

septomları bastırılır.

Bu davranış, asıl neden biliniyorken ikincil hata ve kapasite alarmlarının Slack kanalını doldurmasını engeller.

8. Alarm kataloğu

Uygulama alarmları

Alarm	Severity	Koşul	for	keep_firing_for
BackendDown	critical	Backend scrape hedefi <code>up == 0</code> veya tamamen kayıp	2 dk	5 dk
HighHttp5xxRatio	warning	Son 5 dk içinde en az 20 istek ve 5xx oranı <code>%5</code> üzeri	5 dk	5 dk
HighHttp5xxRatio	critical	Son 5 dk içinde en az 20 istek ve 5xx oranı <code>%20</code> üzeri	5 dk	5 dk
HikariPoolSaturation	warning	Active/max <code>%90</code> üzeri veya pending bağlantı var	5 dk	5 dk
HighJvmHeapUsage	warning	JVM heap kullanım oranı <code>%85</code> üzeri	15 dk	5 dk
HighProcessCpu	warning	Backend process CPU kullanımı <code>%85</code> üzeri	15 dk	5 dk

5xx kurallarındaki minimum 20 istek şartı, düşük trafikte tek bir hatanın yüksek yüzde oluşturarak yanlış alarm üretmesini engeller. Oran `%20` üzerine çıktığında warning ve critical aynı anda aktif olabilir; inhibition yalnızca critical bildirimin gönderilmesini sağlar.

Monitoring alarmları

Alarm	Severity	Koşul	for	keep_firing_for
AlertmanagerDown	critical	Alertmanager scrape hedefi up == 0 veya kayıp	2 dk	5 dk
AlertmanagerNotificationFailures	critical	Son 5 dk içinde bildirim gönderim hatası	2 dk	5 dk
PrometheusRuleEvaluationFailures	warning	Son 5 dk içinde rule evaluation hatası	2 dk	5 dk
PrometheusAlertDeliveryFailures	critical	Prometheus'un Alertmanager'a alarm gönderim hatası	2 dk	5 dk
PrometheusConfigReloadFailed	warning	Son Prometheus config reload başarısız	5 dk	Yok
AlertmanagerConfigReloadFailed	warning	Son Alertmanager config reload başarısız	5 dk	Yok

Prometheus veya Alertmanager'ın kendi alarm yolundaki arızalar Slack'e ulaşamayabilir. Production'da bu hattı dışarıdan izleyen heartbeat/dead-man's switch servisi kullanılması bu nedenle önerilir.

9. Slack entegrasyonu ve secret yönetimi

Webhook oluşturma

- Slack API üzerinde **Finovation Alerts** isimli Blank App oluşturulur.
- App için Incoming Webhooks etkinleştirilir.
- Add New Webhook to Workspace seçilir.
- Hedef kanal olarak #alerts yetkilendirilir.
- Üretilen <https://hooks.slack.com/services/...> URL'si kopyalanır.

Secret dosyasını hazırlama

Proje kökünde:

```
Copy-Item monitoring\alertmanager\secrets\slack_webhook_url.example
monitoring\alertmanager\secrets\slack_webhook_url
notepad monitoring\alertmanager\secrets\slack_webhook_url
```

Örnek değer gerçek webhook URL'siyle değiştirilir. Dosyada yalnızca URL bulunmalıdır.

Gerçek secret dosyası **.gitignore** kapsamındadır ve Docker Compose tarafından container içindeki aşağıdaki konuma bağlanır:

```
/run/secrets/alertmanager_slack_webhook_url
```

Alertmanager config'i URL'yi doğrudan içermez; **api_url_file** aracılığıyla bu dosyayı okur.

Webhook URL'si parola gibi değerlendirilmelidir. Chat, issue, log veya Git üzerinden paylaşılmamalıdır. Sızması durumunda Slack'ten hemen iptal edilip yeni webhook oluşturulmalıdır.

10. Kurulum ve başlatma

Ön koşullar

- Docker Desktop çalışıyor olmalıdır.
- Repository kökünde geçerli `.env` dosyası bulunmalıdır.
- Slack webhook secret dosyası hazırlanmış olmalıdır.

Servisleri başlatma

```
docker compose up -d alertmanager prometheus grafana
```

Çalışma durumunu kontrol etme

```
docker compose ps alertmanager prometheus grafana backend
```

Beklenen durum ilgili servislerin **Up** olmasıdır.

Alertmanager readiness kontrolü:

```
Invoke-WebRequest -UseBasicParsing http://localhost:9093/-/ready
```

Beklenen yanıt **200 OK** değeridir.

11. Doğrulama ve test

Compose şeması

```
docker compose config --quiet
```

Prometheus config ve rule kontrolü

```
docker compose run --rm --no-deps --entrypoint /bin/promtool prometheus check config  
/etc/prometheus/prometheus.yml
```

Beklenen sonuç iki rule dosyasının ve toplam 12 kuralın başarılı biçimde doğrulanmasıdır.

Prometheus rule unit testleri

```
docker compose run --rm --no-deps --entrypoint /bin/promtool prometheus test rules  
/etc/prometheus/tests/application.rules.test.yml
```

Mevcut testler şunları doğrular:

- Backend iki dakika erişilemez kaldığında **BackendDown** firing olur.
- Sürekli **%5** üzeri 5xx oranı warning alarmı oluşturur.

Alertmanager config ve template kontrolü

```
docker compose run --rm --no-deps --entrypoint /bin/amtool alertmanager check-config  
/etc/alertmanager/alertmanager.yml
```

Beklenen sonuç route, iki receiver, iki inhibition rule ve Slack template'in başarıyla doğrulanmasıdır.

Uçtan uca Slack testi

```
docker compose exec alertmanager amtool --alertmanager.url=http://localhost:9093 alert add  
FinovationSlackTest severity=warning environment=local service=finovation-monitoring  
component=alertmanager --annotation=summary="Slack entegrasyonu çalışıyor"
```

Warning route'un `group_wait` ayarı nedeniyle mesajın `#alerts` kanalına ulaşması yaklaşık 30 saniye sürebilir. Prometheus dışından tek sefer gönderilen test alarmı `resolve_timeout` sonrasında çözülür ve resolved bildirimi üretilir.

12. Günlük işletim

UI adresleri

- Alertmanager: `http://localhost:9093`
- Grafana: `http://localhost:3000`

Log inceleme

```
docker compose logs --tail=100 alertmanager prometheus
```

Canlı takip:

```
docker compose logs -f alertmanager prometheus
```

Prometheus hedeflerini kontrol etme

```
docker compose exec -T prometheus /bin/promtool query instant http://localhost:9090 "up"
```

`finovation-backend`, `prometheus` ve `alertmanager` hedeflerinin her biri için beklenen değer `1`'dir.

Aktif alarmları listeleme

```
docker compose exec alertmanager amtool --alertmanager.url=http://localhost:9093 alert query
```

Planlı bakım ve silence

Planlı bakımda rule dosyasını silmek veya Alertmanager'ı kapatmak yerine Alertmanager UI üzerinden süreli silence oluşturulmalıdır.

Silence oluştururken mümkün olan en dar matcher kullanılmalıdır:

```
environment="local"  
service="finovation-backend"
```

Silence açıklamasında bakım nedeni, sorumlu kişi ve ilgili iş kaydı belirtilmelidir.

13. Değişiklik yönetimi

Alarm kuralı değiştirirken

- İlgili `.rules.yml` dosyası düzenlenir.
- `promtool check config` çalıştırılır.
- Etkilenen davranış için rule unit testi eklenir veya güncellenir.
- `promtool test rules` çalıştırılır.
- Prometheus yeniden başlatılır.
- Prometheus logları ve aktif rule durumu kontrol edilir.

```
docker compose restart prometheus  
docker compose logs --tail=100 prometheus
```

Alertmanager route veya template değiştirirken

1. `alertmanager.yml` veya `templates/slack.tmpl` düzenlenir.
2. `amtool check-config` çalıştırılır.
3. Alertmanager yeniden başlatılır.
4. Sentetik Slack testi gönderilir.

```
docker compose restart alertmanager
docker compose logs --tail=100 alertmanager
```

Config doğrulaması başarısızken servis yeniden başlatılmamalıdır.

14. Sorun giderme

Belirti	Olası neden	Kontrol / çözüm
Slack mesajı gelmiyor	Webhook secret eksik veya geçersiz	Secret dosyasının varlığını ve tek satır URL içerdiğini kontrol edin.
Slack 403/404 hatası	Webhook iptal edilmiş veya yanlış workspace'e ait	Slack App ayarından yeni webhook oluşturun.
Alertmanager başlamıyor	Config, template veya secret dosyası hatalı	<code>amtool check-config</code> ve Alertmanager loglarını çalıştırın.
Prometheus başlamıyor	PromQL veya YAML hatası	<code>promtool check config</code> çalıştırın.
Alarm Prometheus'ta var ama Slack'te yok	Severity route ile eşleşmiyor, silence veya inhibition aktif	Alarm etiketlerini ve Alertmanager UI durumunu inceleyin.
Aynı alarm sürekli tekrarlanıyor	Koşul devam ediyor veya etiketler her istekte değişiyor	Repeat interval ve metric cardinality değerlerini kontrol edin.
Resolved mesajı gelmiyor	Alarm hâlâ firing veya receiver ayarı değişmiş	Prometheus <code>ALERTS</code> serisini ve <code>send_resolved</code> ayarını kontrol edin.
Backend çalışıyor ama <code>BackendDown</code> firing	Management portu veya <code>/actuator/prometheus</code> erişilemiyor	Backend logları, <code>8081</code> portu ve Prometheus target durumunu kontrol edin.
Alertmanager UI açılmıyor	Container kapalı veya port başka süreçte	<code>docker compose ps</code> ve <code>localhost:9093</code> portunu kontrol edin.

Önerilen teshis sırası

```
docker compose ps alertmanager prometheus backend
Invoke-WebRequest -UseBasicParsing http://localhost:9093/-/ready
docker compose logs --tail=100 alertmanager prometheus
docker compose exec -T prometheus /bin/promtool query instant http://localhost:9090 "up"
```

15. Güvenlik notları

- Slack webhook URL'si repository'ye commit edilmez.
- Alertmanager UI yalnızca loopback adresine publish edilir.
- Backend management portu host'a publish edilmez; yalnızca Compose ağı içinden

erişilir.

- Alert annotation ve metric label'larında kullanıcı adı, e-posta, IP, token,

user ID veya ham exception mesajı kullanılmamalıdır.

- Webhook sızıntısında eski webhook iptal edilmeli, yeni secret yerel dosyaya

yazılmalı ve Alertmanager yeniden başlatılmalıdır.

- Production'da Alertmanager UI uzaktan erişilecekse TLS, kimlik doğrulama ve

ağ erişim kontrolü uygulanmalıdır.

16. Kalıcılık ve veri davranışı

- Prometheus metrik geçmişi `finovation_prometheus_data` volume'ünde tutulur.
- Alertmanager silence ve notification log verileri

`finovation_alertmanager_data` volume'ünde tutulur.

- Grafana ayar/verileri `finovation_grafana_data` volume'ünde tutulur.
- Alertlerin kaynağı Prometheus'tur; Alertmanager yeniden başladığında aktif

alarmlar Prometheus tarafından tekrar gönderilir.

Volume'lerin silinmesi geçmiş metriklerin, silence kayıtlarının veya Grafana verilerinin kaybına yol açabilir. Operasyon sırasında volume silme komutları kullanılmamalıdır.

17. Mevcut sınırlamalar

1. Alertmanager ve Prometheus aynı Docker host'unda çalışır; host tamamen

kapanırsa bu sistem Slack bildirimi gönderemez.

1. Tek Alertmanager instance'ı vardır; gerçek HA sağlanmamaktadır.
2. `environment` etiketi şu an sabit olarak `local` değerindedir.
3. Prometheus ve Grafana image'ları `latest` tag'i kullanır; sürümler sabitlenmiş

değildir.

1. Frontend kullanılabilirliği bağımsız bir HTTP probe ile izlenmemektedir.
2. Redis ve SQL Server Docker healthcheck'leri Prometheus alarmına dönüşmez.
3. Login, rate-limit, password reset ve e-posta teslimi için özel Micrometer

sayaçları henüz yoktur.

1. p95/p99 HTTP latency alarmı için histogram bucket'ları henüz etkin değildir.
2. Slack dışında on-call/pager receiver bulunmamaktadır.

18. Production'a geçiş kontrol listesi

- [] `environment` değerini `staging` veya `production` olarak ayarla.
- [] Prometheus ve Grafana image sürümlerini test edilmiş versiyonlara sabitle.
- [] Secret dosyasını production secret manager/orchestrator mekanizmasına taşı.
- [] Alertmanager ve Prometheus'u farklı failure domain'lerde yedekli çalıştır.
- [] Harici uptime/heartbeat servisiyle alarm hattını dışarıdan izle.
- [] Critical alarmlar için gerçek on-call/pager receiver ekle.

- [] Frontend/API için Blackbox Exporter probe'ları ekle.
- [] Redis ve SQL Server için uygulama health metriği veya exporter ekle.
- [] HTTP p95/p99 histogramlarını ve latency alarmlarını ekle.
- [] Auth, rate-limit, password reset ve mail delivery custom metriklerini ekle.
- [] Alarm annotation'larına erişilebilir runbook URL'leri ekle.
- [] Rule/config doğrulamalarını CI pipeline'a ekle.
- [] Prometheus retention, CPU ve memory limitlerini kapasiteye göre ayarla.
- [] Alertmanager UI için TLS, authentication ve network policy uygula.
- [] Eşik değerlerini production trafik baseline'ına göre yeniden kalibre et.

19. Bildirimleri geçici durdurma ve geri dönüş

Kısa süreli bakımda önerilen yöntem Alertmanager silence kullanmaktır. Slack bildirimlerini kalıcı olarak değiştirmek gerekiyorsa route önce `null` receiver'a yönlendirilip `amtool check-config` ile doğrulanmalıdır.

Entegrasyon kodunu tamamen geri almak gerektiğinde değişiklikler version control üzerinden revert edilmeli, ardından Compose servisleri yeniden uygulanmalıdır. Alertmanager volume'ü otomatik olarak silinmemelidir; silence ve notification geçmişinin korunması tercih edilir.

20. Kabul testi sonuçları

4 Ağustos 2026 tarihinde aşağıdaki kontroller başarılı tamamlanmıştır:

- Docker Compose şeması geçerli.
- Prometheus config geçerli; iki rule dosyası ve 12 alarm yüklendi.
- Prometheus rule unit testleri başarılı.
- Alertmanager config, iki receiver, iki inhibition rule ve Slack template geçerli.
- Alertmanager readiness endpoint'i `200 OK` döndü.
- Backend, Prometheus ve Alertmanager scrape hedeflerinin tamamı `up = 1` döndü.
- Sentetik warning alarmı Slack `#alerts` kanalına ulaştı.
- Prometheus ve Alertmanager loglarında config yükleme hatası görülmedi.

21. Resmî referanslar

- [Prometheus alerting overview](#)
- [Prometheus alerting rules](#)
- [Prometheus rule unit testing](#)
- [Alertmanager configuration](#)
- [Alertmanager high availability](#)
- [Slack Incoming Webhooks](#)